

INFORME TÉCNICO: IMPLEMENTACIÓN DE ALGORITMOS DE RECORTE

Asignatura: Computación Gráfica

Nombre: Josue Andel Chango Parra

Contenido

1. Introducción	¡Error! Marcador no definido.
2. Algoritmos de Recorte de Líneas.....	¡Error! Marcador no definido.
Cohen-Sutherland	¡Error! Marcador no definido.
Liang-Barsky	¡Error! Marcador no definido.
Recorte Paramétrico.....	¡Error! Marcador no definido.
3. Algoritmos de Recorte de Polígonos.....	¡Error! Marcador no definido.
Sutherland-Hodgman.....	¡Error! Marcador no definido.
Vertex Clipping.....	¡Error! Marcador no definido.
Bounding Box Clipping.....	¡Error! Marcador no definido.
4. Comparación entre Algoritmos	¡Error! Marcador no definido.
5. Conclusiones.....	¡Error! Marcador no definido.

1. Introducción

El recorte (clipping) es una operación fundamental en los sistemas de gráficos por computadora que consiste en determinar qué partes de una figura geométrica son visibles dentro de una región rectangular denominada ventana de recorte, descartando todo lo que queda fuera de ella. Este proceso es esencial en cualquier pipeline gráfico moderno, ya que garantiza que únicamente los elementos dentro del área de visualización sean procesados, evitando cómputo innecesario.

La ventana de recorte se define mediante cuatro límites: x_{min} , x_{max} , y_{min} e y_{max} . Cualquier segmento de línea cuyos extremos se encuentren fuera de estos límites debe ser parcialmente o totalmente descartado antes de su visualización. De manera análoga, un polígono puede quedar reducido a un polígono de menor número de vértices tras el recorte.

El presente informe documenta la implementación en C# Windows Forms de seis algoritmos clásicos de recorte, clasificados en dos categorías: algoritmos de recorte de líneas (Cohen-Sutherland, Liang-Barsky y Recorte Paramétrico) y algoritmos de recorte de polígonos (Sutherland-Hodgman, Vertex Clipping y Bounding Box Clipping). Para cada algoritmo se presenta su descripción conceptual, funcionamiento paso a paso, fórmulas matemáticas y análisis comparativo.

El objetivo principal del proyecto es comprender los mecanismos internos de cada algoritmo, identificar sus diferencias en eficiencia y precisión, y determinar los escenarios de uso más adecuados para cada uno en el contexto del desarrollo de software gráfico.

2. Algoritmos de Recorte de Líneas

Los algoritmos de recorte de líneas determinan la porción visible de un segmento definido por los extremos $P1(x1, y1)$ y $P2(x2, y2)$ dentro de la ventana de recorte. Un segmento puede estar completamente dentro (se acepta), completamente fuera (se rechaza) o parcialmente dentro (se calcula la intersección con los bordes de la ventana).

Cohen-Sutherland

Descripción del Algoritmo

El algoritmo de Cohen-Sutherland, desarrollado en 1967, asigna un código de región (outcode) de 4 bits a cada extremo del segmento codificando su posición relativa respecto a los cuatro bordes de la ventana. Esto permite determinar eficientemente si un segmento debe aceptarse, rechazarse o recortarse sin calcular intersecciones en los casos triviales.

Funcionamiento Paso a Paso

- Calcular el outcode de $P1$ y $P2$. Cada bit indica si el punto está fuera de un borde: bit 0 = izquierda (x menor a x_{min}), bit 1 = derecha (x mayor a x_{max}), bit 2 = abajo (y menor a y_{min}), bit 3 = arriba (y mayor a y_{max}).
- Aceptación trivial: si $(\text{outcode1 OR outcode2}) = 0000$, ambos puntos están dentro; aceptar el segmento completo.

- Rechazo trivial: si (outcode1 AND outcode2) es diferente de 0000, ambos puntos están en la misma región exterior; rechazar el segmento.
- Si ninguna condición trivial se cumple: seleccionar el extremo con outcode diferente de cero, identificar el borde por el que sale y calcular la intersección con ese borde.
- Reemplazar el extremo exterior por el punto de intersección y repetir el proceso desde el paso 1 con el segmento recortado.

Fórmulas Utilizadas

El outcode de un punto (x, y) se construye bit a bit:

$$\text{bit IZQUIERDA} = (x < x_{\min}) \quad \text{bit DERECHA} = (x > x_{\max})$$

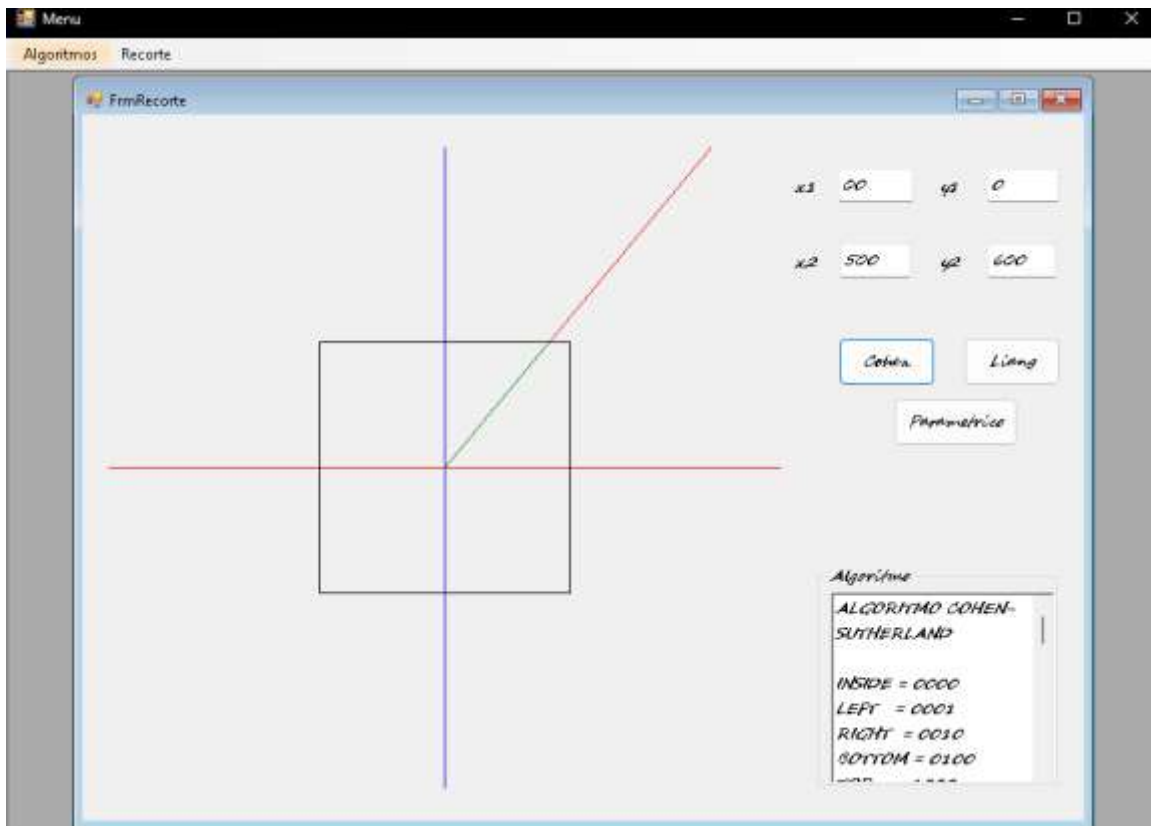
$$\text{bit ABAJO} = (y < y_{\min}) \quad \text{bit ARRIBA} = (y > y_{\max})$$

Intersección del segmento con un borde vertical $x = x_{\text{Borde}}$:

$$y_{\text{interseccion}} = y_1 + \text{pendiente} * (x_{\text{Borde}} - x_1) \quad \text{pendiente} = (y_2 - y_1) / (x_2 - x_1)$$

Intersección con un borde horizontal $y = y_{\text{Borde}}$:

$$x_{\text{interseccion}} = x_1 + (y_{\text{Borde}} - y_1) / \text{pendiente}$$



Liang-Barsky

Descripción del Algoritmo

El algoritmo de Liang-Barsky (1984) reformula el recorte de líneas en términos paramétricos. En lugar de calcular intersecciones por borde de forma iterativa como Cohen-Sutherland, calcula directamente los valores del parámetro t para los que el segmento intersecta cada borde, determinando el intervalo $[t_1, t_2]$ que define la porción visible del segmento en una sola pasada. Esto lo hace más eficiente cuando los segmentos intersectan la ventana.

Funcionamiento Paso a Paso

- Parametrizar el segmento: $P(t) = P_1 + t*(P_2 - P_1)$ para t en $[0,1]$ recorre el segmento de P_1 a P_2 .
- Definir para cada uno de los cuatro bordes los valores p_i y q_i (ver fórmulas).
- Inicializar $t_1 = 0$ y $t_2 = 1$ (intervalo completo del segmento).
- Para cada borde: si $p_i = 0$ y q_i menor a 0, el segmento es paralelo y exterior, rechazar. Si p_i menor a 0, el segmento va del exterior al interior; actualizar $t_1 = \max(t_1, q_i/p_i)$. Si p_i mayor a 0, el segmento va del interior al exterior; actualizar $t_2 = \min(t_2, q_i/p_i)$.
- Si t_1 mayor a t_2 , el segmento es completamente exterior, rechazar. De lo contrario, los extremos visibles son: $P_1' = P(t_1)$ y $P_2' = P(t_2)$.

Fórmulas Utilizadas

Valores p_i y q_i para cada borde de la ventana:

$$p1 = -(x2-x1), \quad q1 = x1 - xmin \quad (\text{borde izquierdo})$$

$$p2 = (x2-x1), \quad q2 = xmax - x1 \quad (\text{borde derecho})$$

$$p3 = -(y2-y1), \quad q3 = y1 - ymin \quad (\text{borde inferior})$$

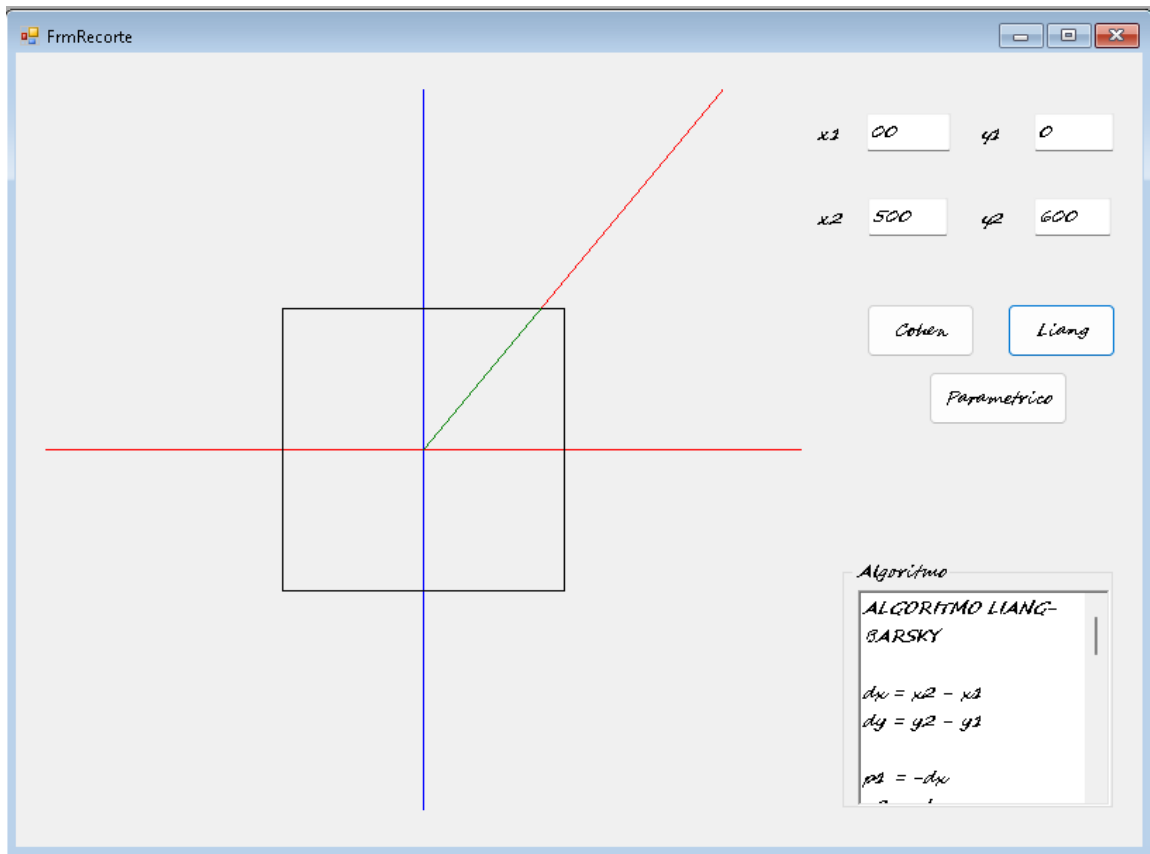
$$p4 = (y2-y1), \quad q4 = ymax - y1 \quad (\text{borde superior})$$

Parámetro de intersección con cada borde:

$$t = q_i / p_i$$

Extremos del segmento recortado evaluando la parametrización:

$$x(t) = x1 + t*(x2-x1) \quad y(t) = y1 + t*(y2-y1)$$



Recorte Paramétrico

Descripción del Algoritmo

El algoritmo de Recorte Paramétrico es una variante generalizada del enfoque paramétrico aplicado al recorte de líneas. Extiende la formulación de Liang-Barsky al concepto de recorte contra una secuencia de semiplanos definidos por inecuaciones, lo que permite aplicarlo a ventanas no rectangulares y a escenarios de recorte tridimensional. En el caso del recorte rectangular plano, opera de forma equivalente a Liang-Barsky pero con una formulación más general basada en el producto punto entre la dirección del segmento y la normal de cada borde.

Funcionamiento Paso a Paso

- Representar el segmento de forma paramétrica: $P(t) = P1 + t \cdot d$, donde $d = P2 - P1$ y t en $[0, 1]$.
- Para cada semiplano de recorte definido por su normal n y un punto q sobre el borde, calcular el parámetro de intersección $t = n \cdot (q - P1) / (n \cdot d)$.
- Clasificar el borde como de entrada o salida: si $n \cdot d$ menor a 0, el segmento entra al semiplano, actualizar $tE = \max(tE, t)$. Si $n \cdot d$ mayor a 0, el segmento sale, actualizar $tL = \min(tL, t)$.

- Si en algún momento t_E mayor a t_L , el segmento está completamente fuera; rechazar.
- Si t_E menor o igual a t_L al procesar todos los bordes, los extremos visibles son $P(t_E)$ y $P(t_L)$.

Fórmulas Utilizadas

Dirección del segmento:

$$d = (dx, dy) = (x_2 - x_1, y_2 - y_1)$$

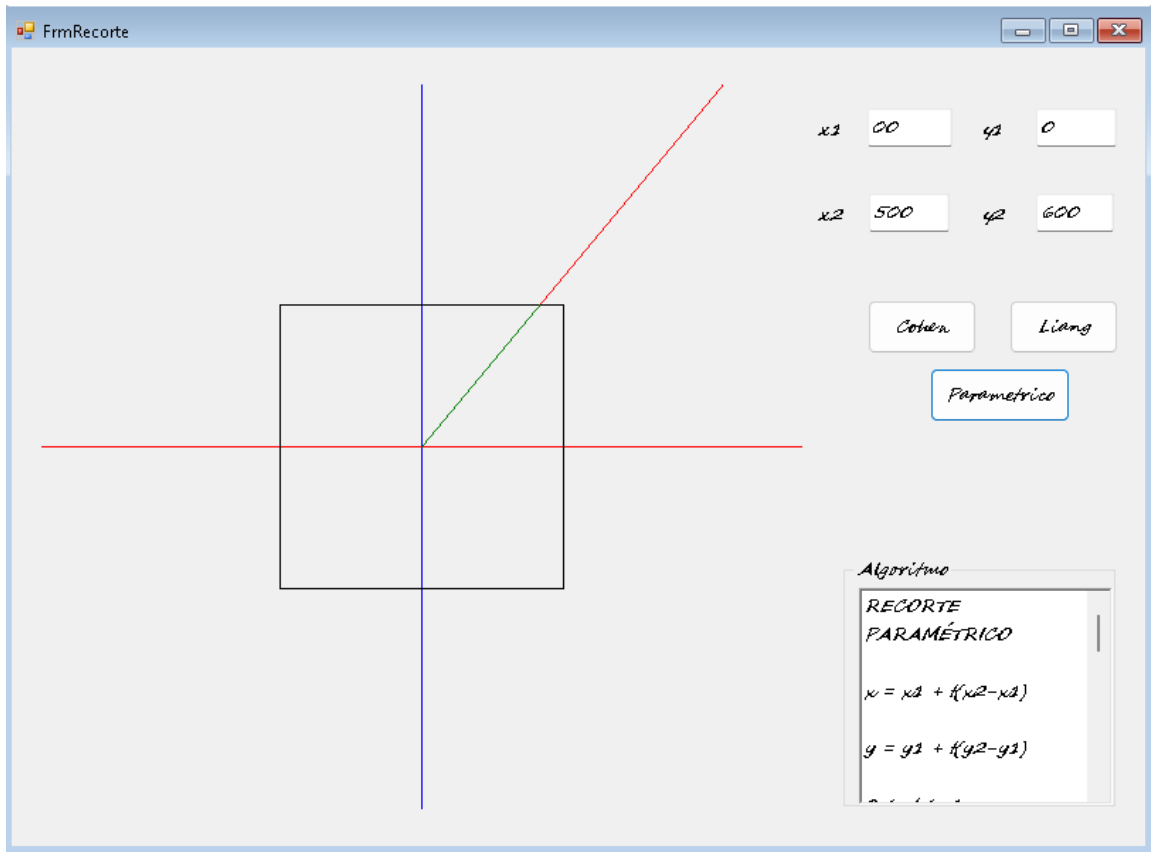
Parámetro de intersección con un semiplano de normal n y punto de referencia q :

$$t = n \cdot (q - P_1) / (n \cdot d)$$

Criterio de entrada o salida al semiplano:

$$n \cdot d \text{ menor a } 0 \Rightarrow \text{punto de entrada: } t_E = \max(t_E, t)$$

$$n \cdot d \text{ mayor a } 0 \Rightarrow \text{punto de salida: } t_L = \min(t_L, t)$$



3. Algoritmos de Recorte de Polígonos

El recorte de polígonos es más complejo que el de líneas, ya que al recortar un polígono contra una ventana rectangular el resultado puede ser otro polígono con distinto número de vértices. Los algoritmos deben gestionar la adición o eliminación de vértices y el cierre correcto del contorno resultante.

Sutherland-Hodgman

Descripción del Algoritmo

El algoritmo de Sutherland-Hodgman (1974) recorta un polígono sucesivamente contra cada uno de los cuatro bordes de la ventana, uno a la vez. El resultado del recorte contra un borde se utiliza como entrada para el recorte contra el siguiente. Este enfoque pipeline simplifica el problema reduciéndolo a resolver cuatro veces el caso más simple: recortar una lista de vértices contra un único semiplano.

Funcionamiento Paso a Paso

- Inicializar la lista de vértices de salida con los vértices originales del polígono de entrada.
- Para cada uno de los cuatro bordes de la ventana (izquierdo, derecho, inferior, superior): usar la lista actual como entrada y construir una nueva lista de salida.
- Para cada par de vértices consecutivos (S, P) en la lista de entrada, aplicar las cuatro reglas del algoritmo según su posición respecto al borde actual:
- Regla 1: S fuera, P dentro: agregar intersección y luego P.
- Regla 2: S dentro, P dentro: agregar solo P.
- Regla 3: S dentro, P fuera: agregar solo la intersección.
- Regla 4: S fuera, P fuera: no agregar nada.
- La lista de salida tras el cuarto borde es el polígono recortado final.

Fórmulas Utilizadas

Intersección del segmento S-P con borde vertical $x = x_{\text{Borde}}$:

$$t = (x_{\text{Borde}} - S_x) / (P_x - S_x)$$

$$y_{\text{inters}} = S_y + t * (P_y - S_y)$$

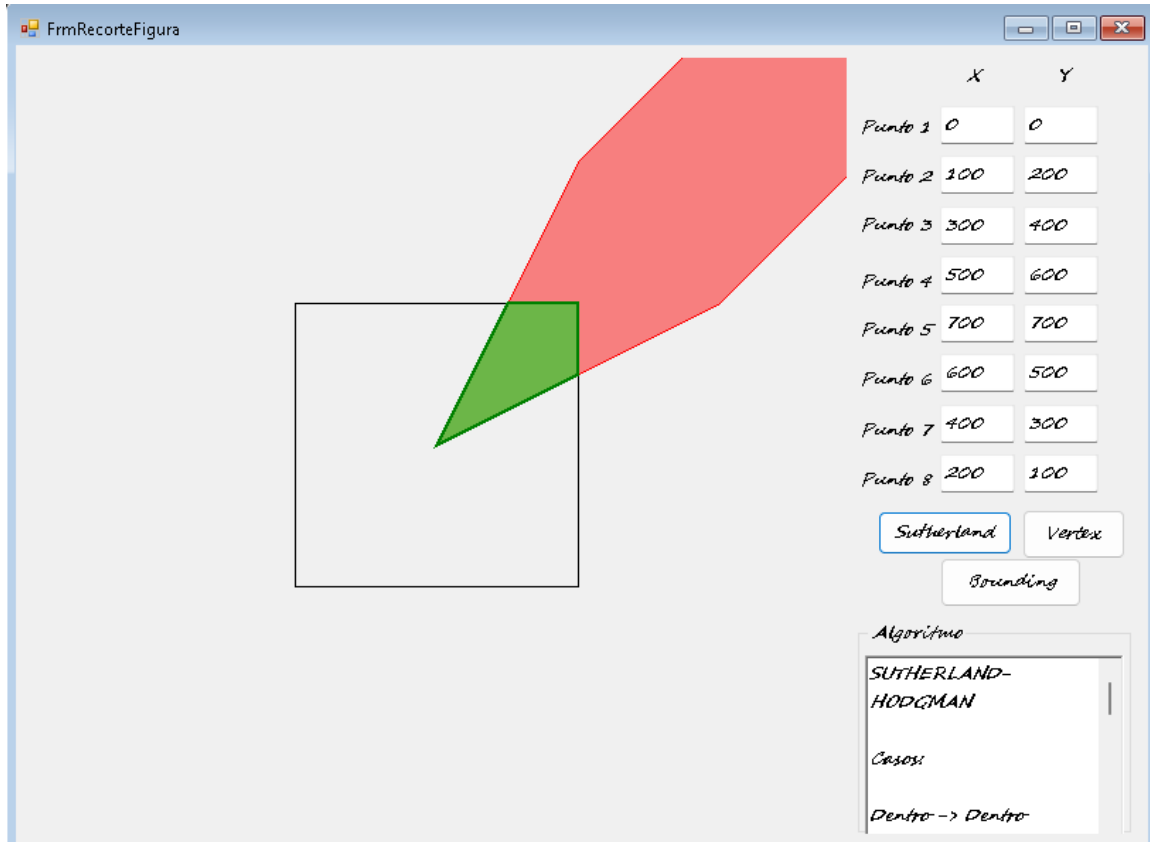
Intersección con borde horizontal $y = y_{\text{Borde}}$:

$$t = (y_{\text{Borde}} - S_y) / (P_y - S_y)$$

$$x_{\text{inters}} = S_x + t * (P_x - S_x)$$

Condición de interior para cada borde:

$$\text{Izquierdo: } x \geq x_{\min} \quad \text{Derecho: } x \leq x_{\max} \quad \text{Inferior: } y \geq y_{\min} \quad \text{Superior: } y \leq y_{\max}$$



Vertex Clipping

Descripción del Algoritmo

El algoritmo de Vertex Clipping opera directamente sobre los vértices del polígono, clasificándolos individualmente como interiores o exteriores a la ventana de recorte sin calcular intersecciones. El resultado retiene únicamente los vértices que caen dentro de la ventana, lo que puede producir polígonos con geometría incorrecta cuando el polígono intersecta parcialmente la ventana. Se utiliza principalmente como filtrado rápido o en aplicaciones donde la precisión exacta no es crítica.

Funcionamiento Paso a Paso

- Recorrer la lista de vértices del polígono uno a uno.
- Para cada vértice (x, y), verificar si satisface las cuatro condiciones de la ventana: $x_{min} \leq x \leq x_{max}$ y $y_{min} \leq y \leq y_{max}$.
- Si el vértice está dentro, agregarlo a la lista de salida.
- Si el vértice está fuera, descartarlo sin calcular ninguna intersección.

- El polígono resultante conecta en orden los vértices de la lista de salida, cerrando el último con el primero.

Fórmulas Utilizadas

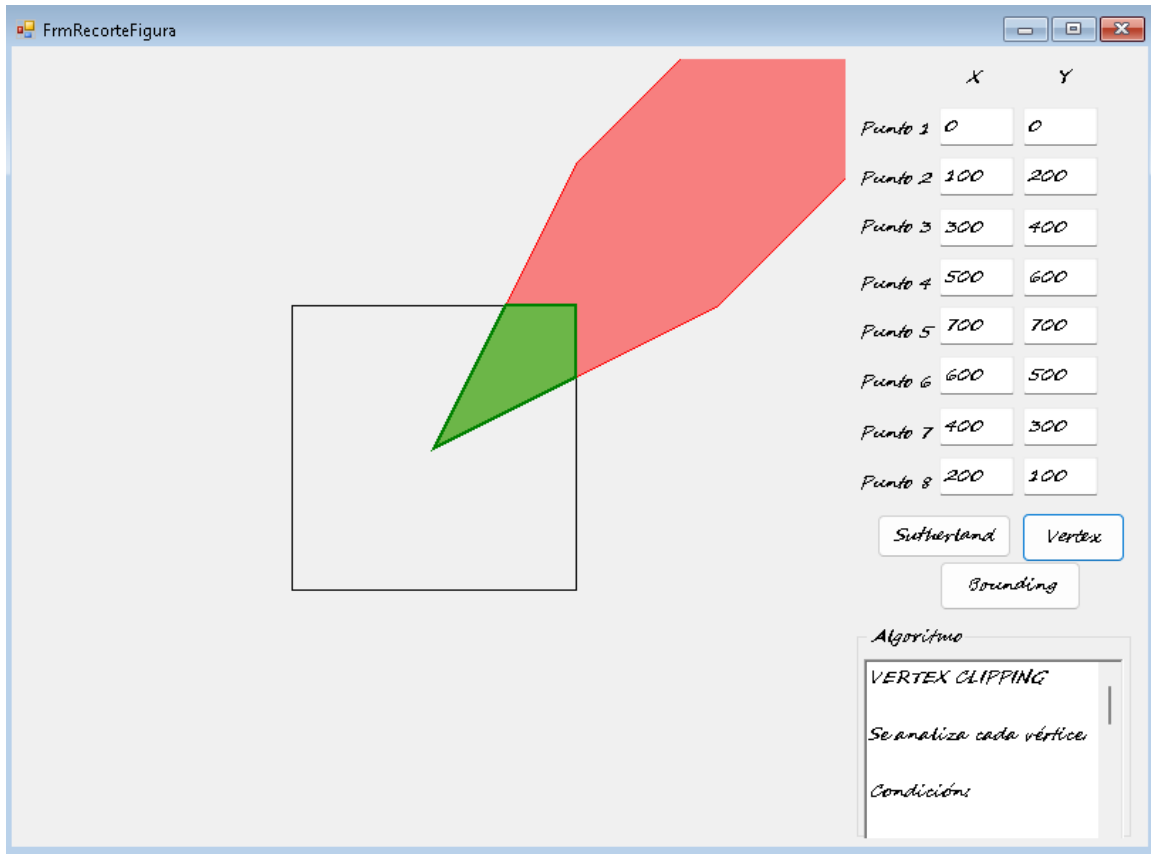
Condición de interior de un vértice (x, y):

$$interior = (x \geq xmin) \text{ AND } (x \leq xmax) \text{ AND } (y \geq ymin) \text{ AND } (y \leq ymax)$$

La lista de salida se construye por filtrado directo, sin cálculo de intersecciones:

Si interior(vi) = verdadero => agregar vi a la lista de salida

Si interior(vi) = falso => descartar vi



Bounding Box Clipping

Descripción del Algoritmo

El algoritmo de Bounding Box Clipping calcula el rectángulo mínimo que contiene al polígono (bounding box) y lo compara con la ventana de recorte para tomar una decisión rápida de aceptación o rechazo total. Si las cajas no se intersectan, el polígono se descarta íntegramente. Si la caja del polígono está completamente dentro de la ventana, se acepta sin más cálculo. Este método es valioso como etapa de prefiltering en pipelines gráficos con grandes cantidades de polígonos, eliminando rápidamente los casos triviales.

Funcionamiento Paso a Paso

- Calcular el bounding box del polígono: $xPmin = \min(x_i)$, $xPmax = \max(x_i)$, $yPmin = \min(y_i)$, $yPmax = \max(y_i)$ sobre todos los vértices.
- Rechazo trivial: si $xPmax$ menor a $xmin$, o $xPmin$ mayor a $xmax$, o $yPmax$ menor a $ymin$, o $yPmin$ mayor a $ymax$, el polígono está completamente fuera; rechazar.

- Aceptación trivial: si $xPmin$ mayor o igual a $xmin$, y $xPmax$ menor o igual a $xmax$, y $yPmin$ mayor o igual a $ymin$, y $yPmax$ menor o igual a $ymax$, el polígono está completamente dentro; aceptar.
- Si ninguna condición trivial se cumple, aplicar un algoritmo de recorte preciso (como Sutherland-Hodgman) para el resultado exacto.

Fórmulas Utilizadas

Cálculo del bounding box del polígono con n vértices:

$$xPmin = \min(x0..xn-1) \quad xPmax = \max(x0..xn-1)$$

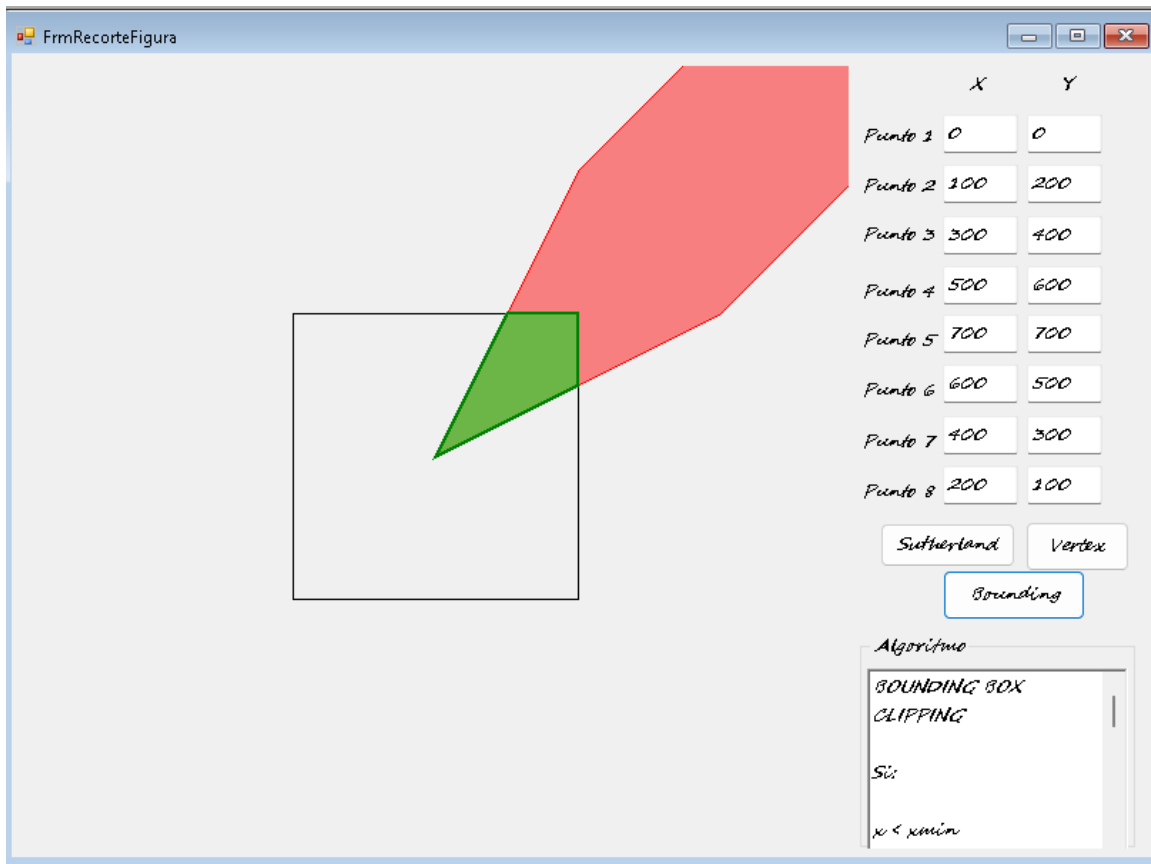
$$yPmin = \min(y0..yn-1) \quad yPmax = \max(y0..yn-1)$$

Condición de no intersección entre el bounding box del polígono y la ventana de recorte:

$$rechazo = (xPmax < xmin) \text{ OR } (xPmin > xmax) \text{ OR } (yPmax < ymin) \text{ OR } (yPmin > ymax)$$

Condición de contenimiento total del polígono dentro de la ventana:

$$aceptacion = (xPmin \geq xmin) \text{ AND } (xPmax \leq xmax) \text{ AND } (yPmin \geq ymin) \text{ AND } (yPmax \leq ymax)$$



4. Comparacion entre Algoritmos

4.1 Algoritmos de Recorte de Lineas

La siguiente tabla compara los tres algoritmos de recorte de lineas en funcion de sus caracteristicas tecnicas mas relevantes:

Criterio	Cohen-Sutherland	Liang-Barsky	Rec. Parametrico
Tipo de operaciones	Enteras (outcode)	Flotante (division)	Flotante (prod. punto)
Rechazo/aceptacion trivial	Muy eficiente	No aplica	No aplica

Eficiencia en intersecciones	Media (iterativo)	Alta (una pasada)	Alta (una pasada)
N. de intersec. calculadas	Hasta 4 por iteracion	Maximo 4 en total	Maximo 4 en total
Complejidad de implementacion	Media	Media	Alta
Generalizacion a 3D	Limitada	Si	Si (semiplanos)
Caso de uso optimo	Muchos rechazos triviales	Frecuentes intersecciones	Ventanas no rectangulares

Tabla 1. Comparativa de algoritmos de recorte de lineas.

Cohen-Sutherland destaca por su capacidad de rechazar o aceptar segmentos trivialmente gracias al outcode, sin calcular ninguna interseccion. Es extremadamente eficiente en escenas con alta proporcion de segmentos completamente fuera o dentro de la ventana. Sin embargo, cuando los segmentos intersectan multiples bordes, debe iterar el proceso varias veces, lo que reduce su ventaja.

Liang-Barsky y el Recorte Parametrico calculan en una sola pasada los parametros t_1 y t_2 que definen directamente los extremos del segmento visible, sin iteraciones. La diferencia entre ambos radica en su generalidad: el Recorte Parametrico puede adaptarse a ventanas definidas por semiplanos arbitrarios y es la base conceptual del recorte tridimensional.

4.2 Algoritmos de Recorte de Poligonos

La siguiente tabla compara los algoritmos de recorte de poligonos en cuanto a precision, rendimiento y adecuacion segun el caso de uso:

Criterio	Sutherland-Hodgman	Vertex Clipping	Bounding Box
Precision geometrica	Muy alta	Baja (sin intersec.)	Alta (con refinamiento)
Velocidad en escenas grandes	Media	Alta	Muy alta (prefilter)

Calcula intersecciones	Si, exactas	No	No (solo caja)
Resultado siempre correcto	Si	Solo si todos dentro	Solo si todos dentro
Complejidad de implementacion	Media-Alta	Baja	Baja
Uso como prefilter	No	Si (aproximado)	Si (exacto)
Caso de uso optimo	Recorte preciso	Filtrado rapido aprox.	Eliminacion temprana

Tabla 2. Comparativa de algoritmos de recorte de poligonos.

Sutherland-Hodgman es el unico que produce siempre un poligono recortado geometricamente correcto, incluyendo poligonos concavos y con multiples intersecciones por borde. Su enfoque pipeline es elegante y generalizable, aunque puede ser costoso para poligonos muy complejos.

Vertex Clipping ofrece la implementacion mas simple pero al costo de precision: al no calcular intersecciones, los poligonos parcialmente dentro de la ventana se representan incorrectamente. Es util unicamente como heuristica de prefiltering aproximada.

Bounding Box Clipping brilla como etapa de prefiltering: con un numero minimo de comparaciones descarta completamente poligonos que no intersectan la ventana. En escenas con muchos poligonos fuera del area visible, esta optimizacion puede representar una mejora de rendimiento de varios ordenes de magnitud.

5. Conclusiones

A partir de la implementacion experimental y el analisis comparativo de los seis algoritmos de recorte documentados, se derivan las siguientes conclusiones tecnicas:

1. La implementación de algoritmos de recorte demostró ser un paso vital para mejorar el rendimiento gráfico, evitando que la CPU y la GPU procesen información geométrica que finalmente no será visible para el usuario final.
2. A nivel de líneas, se constató empíricamente que el algoritmo de Liang-Barsky es matemáticamente más eficiente que el de Cohen-Sutherland, ya que minimiza la necesidad de recalculer puntos mediante el uso de ecuaciones paramétricas y reduce la cantidad de divisiones computacionales.

3. En cuanto a las figuras complejas, Sutherland-Hodgman demostró ser una solución robusta e iterativa, procesando aristas en cascada. Sin embargo, su complejidad computacional escala con el número de vértices, lo que hace indispensable el uso previo de técnicas de optimización.

4. La implementación de Bounding Box Clipping se reveló no como un sustituto, sino como el complemento ideal a cualquier motor gráfico. Realizar una validación de caja delimitadora previa previene la ejecución innecesaria de algoritmos costosos como el Recorte paramétrico o Sutherland-Hodgman sobre entidades completamente fuera de pantalla.